SIMATSENGINEERING(SAVEETHASCHOOLOFENGINEERING)

DepartmentofComputerScienceandEngineering

ASSESSMENTTOOL2–FLOWCHART-TO-CODECONVERSION

CourseCode:	CSA0305	**CourseTitle:**	DataStructures
COAssessed:	CO5–Naturalization(BL4,BL6)	**Assessment:**	AssessmentTool2–Flowchart-To-CodeConversion
Weightage:	35%ofCIA	**TotalMarks:**	100
COSStatement:	*ConstructMSTusing shortest pathalgorithms andtraverse the graphto model and analysereal-world problem.*		
StudentName:	M.VAISHNAV	**Reg. No.:**	192524251
Section/Batch:	2025	**Date:**	04/09/2026

CodeofConduct

I __M.VAISHNAV_____(Name/RegNo) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature ofthe Student: M.VAISHNAV

Instructions

- Thistestcontains5Flowchart-to-codeconversionquestions.Total:100Marks.
- WhenastudentanswerusingFlowchart-to-codeconversionshouldevaluatebasedontheabilityto design clear, logical, and efficient code solutions for data structure problems.
- Nonegativemarking.Unansweredquestionscarryzeromarks.

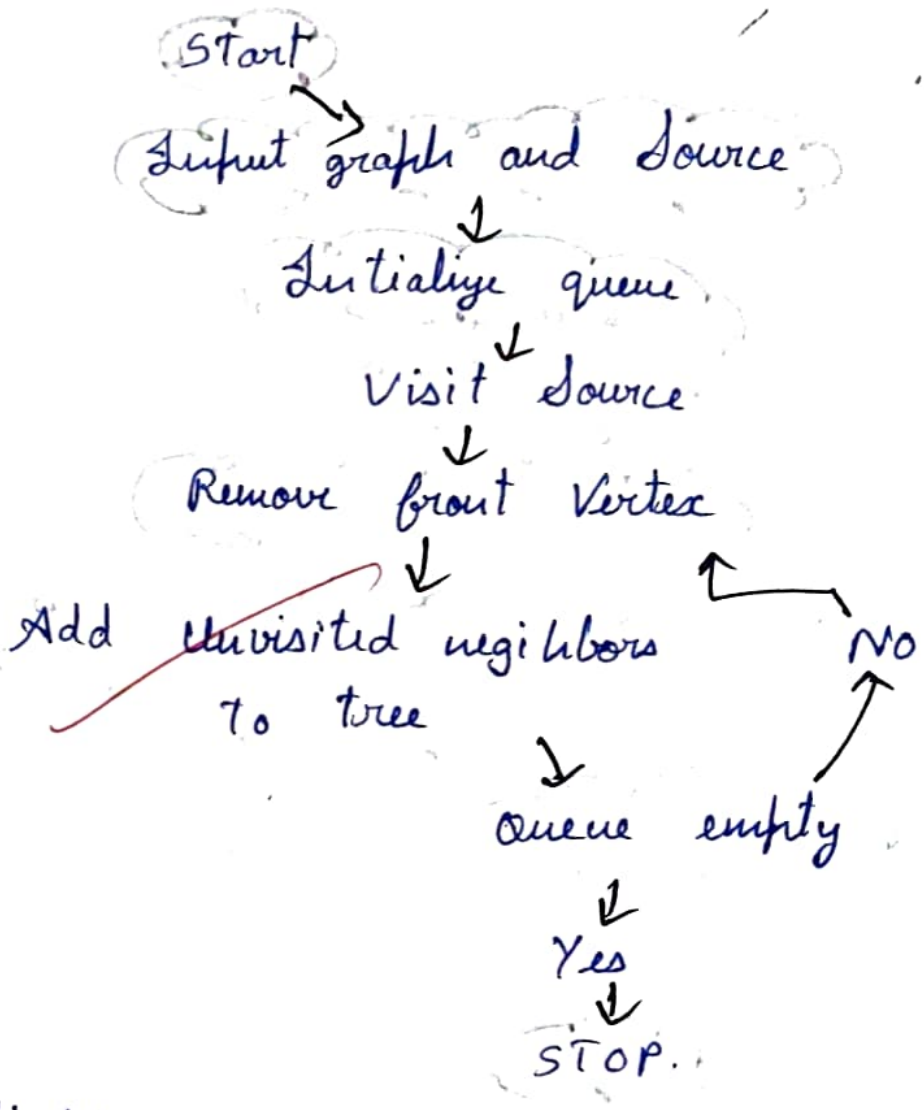
Section/Topic	Focus	QNos	Marks
Graph Traversal	BFS	1	20
ShortestPathAlgorithm	Dijkstra'sAlgorithm	2	20
ShortestPathAlgorithm	UnweightedGraph	3	20
Graph Traversal	BFS	4	20
MinimumSpanningTree	Prim'sAlgorithm	5	20
	GRANDTOTAL:		100Marks

RubricsforCalculation

Criteria	Weightage	Level4–Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
ProblemUnderstanding	20	Demonstratescompleteanddclearunderstanding;handles edge casseffectively	Goodunderstanding;minorgapsinedgecasehandling	Partialunderstanding;missessomerequirements	Misinterpretsproblem,majorgaps
CodeCorrectness	30	Fullycorrect;passesalltestcases	Mostlycorrect;minorerrors	Partiallycorrect;severaltestcasesfail	Incorrectornon-functionalcode
CodeQuality&Readability	20	Clean,modular,well-commented,followsstandards	Readablewithsomestructure	Limitedreadability;poorstructure	Unorganizedandhard to read
ComplexityAnalysis	20	Accuratetime&spacecomplexitywithjustification	Correctbutlimitedexplanation	Incompleteorpartiallycorrect	Missingorincorrect
Testing&Validation	10	Thoroughtestingincludingedgecases	Adequatetesting with few edgecases	Minimaltesting	Notestingevidence

FacultyIn-charge	CourseCoordinator	ProgramDirector
Signature:	Signature:	Signature:
Date:	Date:	Date:

1) Convert the flowchart for generating a BFS tree into code.

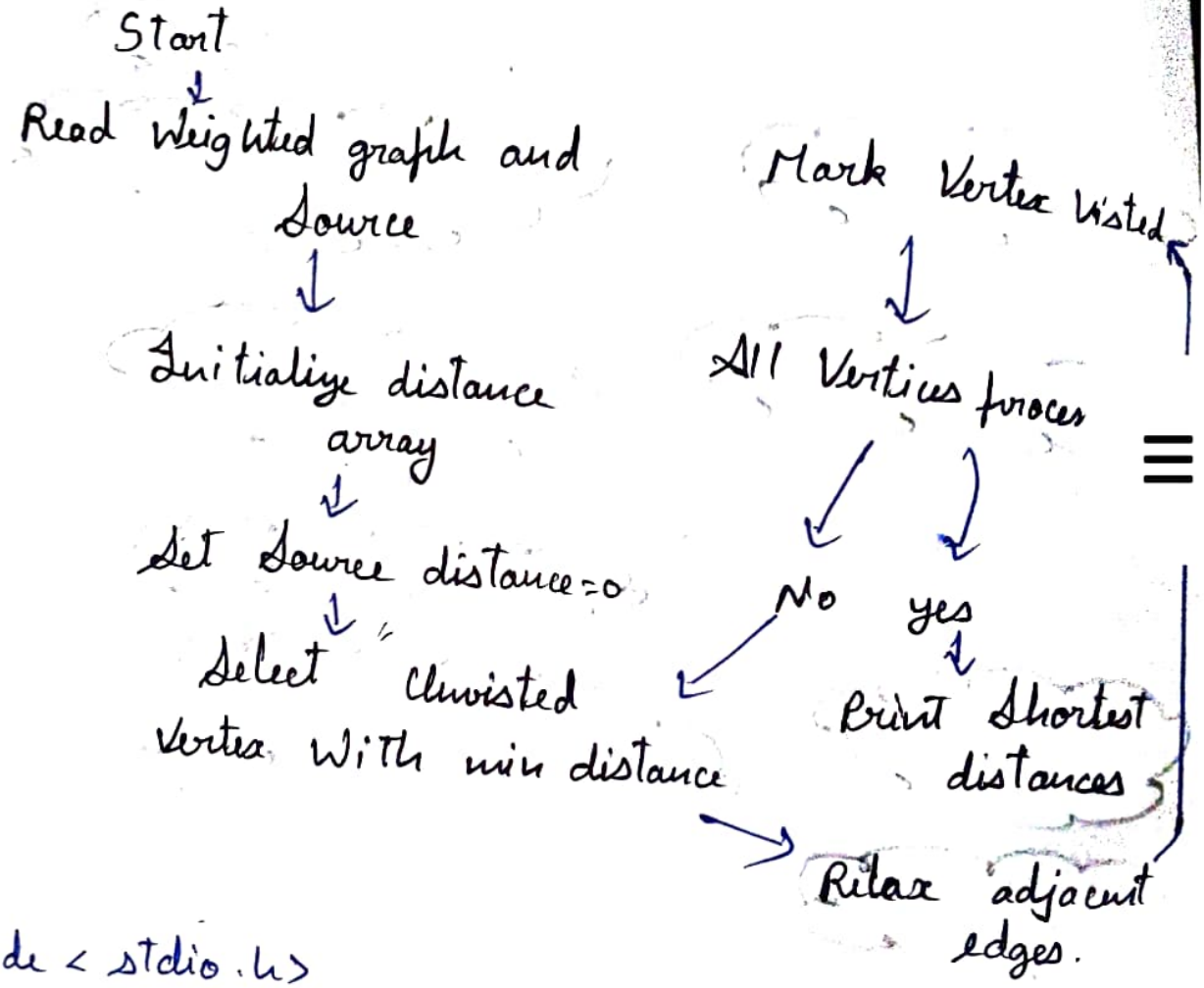


Ans. Code

```
#include <studio.h>
int main(){
int a[10][10], q[10], v[10]={0}, n, s, f=0, r=0, i, j;
scanf("%d", &n);
for(i=0; i<n; i++) for(j=0; j<n; j++) scanf("%d", &a[i][j]);
scanf("%d", &s); q[r++]=s; v[s]=1;
```

```
while (f<r){s=q[f++]; printf("%d",s);
for(i=0;i<n;i++) if(a[s][i] && !v[i] v[i]=1, q[r++]=i;}
}
```

=) Convert the flowchart for Dijkstra's shortest path algorithm into Code.

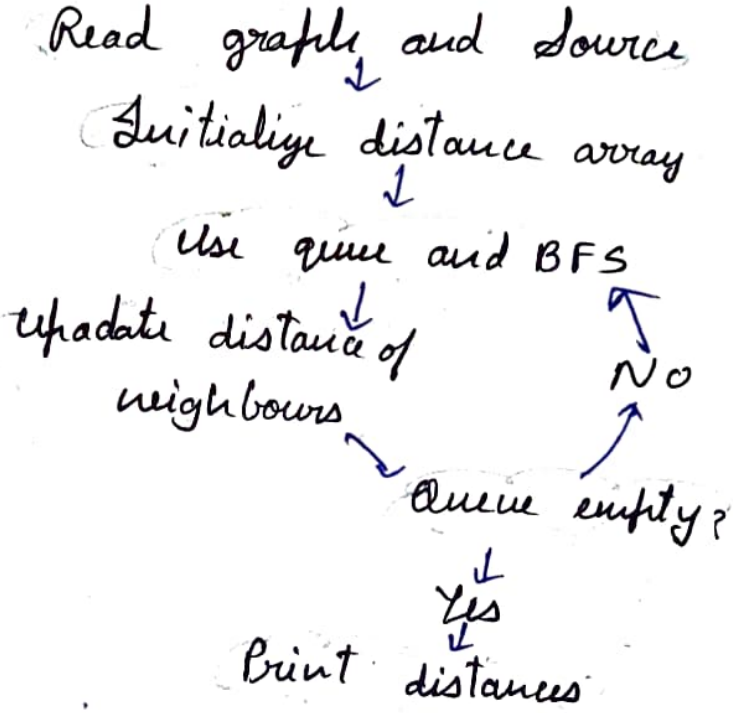


Ans Code

```
#include <stdio.h>
int main(){
int a[10][10],d[10],v[10]={0},n,s,i,j,u,m;
Scanf("%d",&n);
for(i=0;i<n,i++) for(j=0;j<n;j++) scanf("%d",&a[i][j];
scanf("d",&s);
```

```
d[s] = 0
for(i=0; i<u; i++) {m=999;
for(j=0; j<u; j++) if (!v[j] && d[j]<m) m=d[j], u=j;
v[u]=1;
for(j=0; j<u, j++) if( a[u][j] && d[j]>d[u]+a[u]
[j] d[j] = d[u] + a[u][j]; }
for(i=0; i<u; i++) printf("%d", d[i]);
}
```

3) Convert the flowchart for shortest path in an unweighted graph into code.



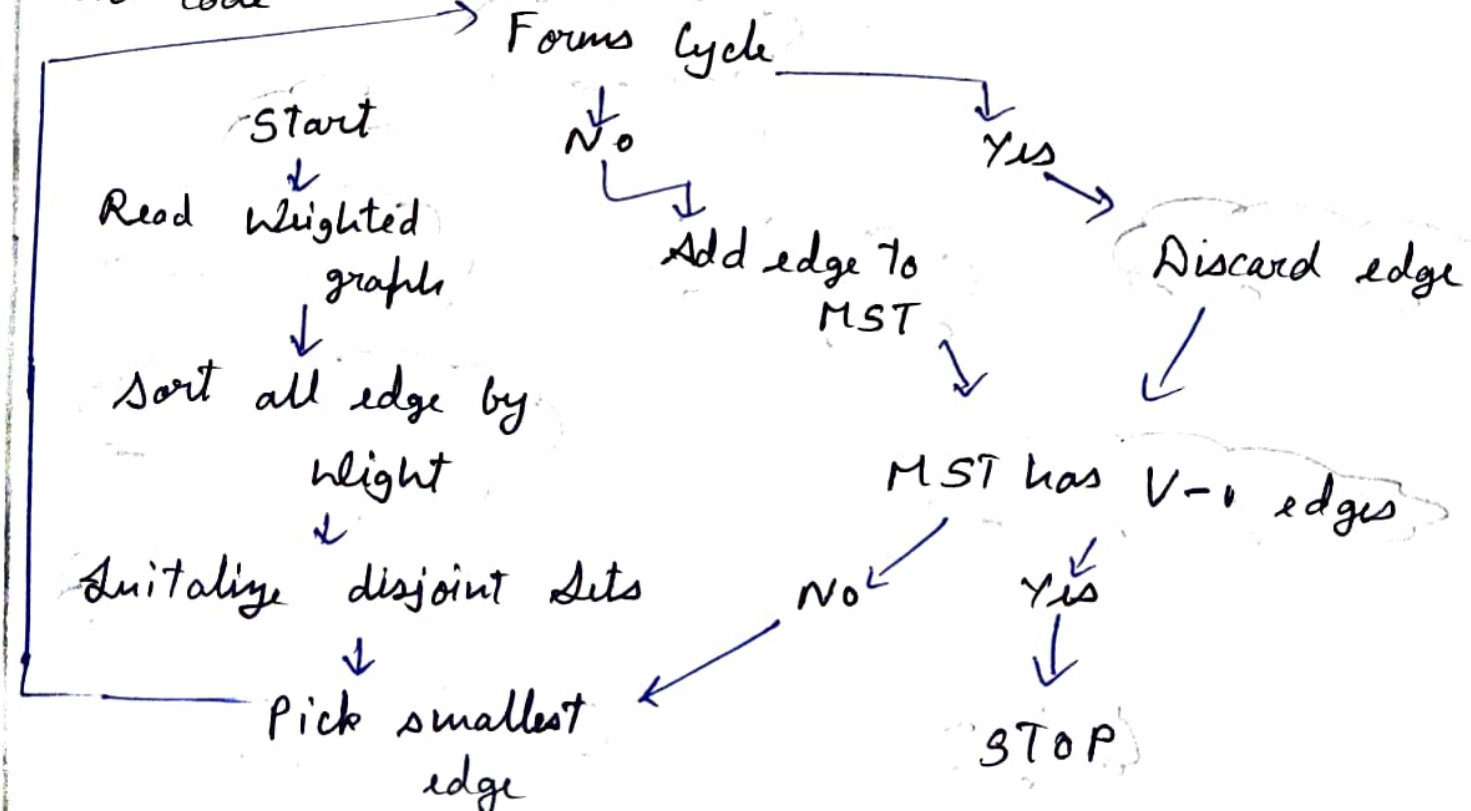
Ans Code

```
#include<stdio.h>
int main(){
int a[10]
[10], d[10], q[10], v[10]={0}, u, s, f=0, r=0, u, i, j;
scanf("%d", &u);
```

Code

```
#include <stdio.h>
int main () {
int a [10] [10], v [10] ={0}, u,i,j, x, m, c=0;
scanf ("%d," &u);
for (i=0 ; i<u; i++) for (j=0; j<u; j
++) scanf ("%d," &a[i][j]);
while (c<u-1) {m= 999;
for (i=0 : j<u; i++) if (v[i] for(j=0; j<u; j++)
if (i v[j] && a[i][j] <m) m=a[i][j], x=j;
v[x] =1; printf ("%d ", m); c++;}
}
```

5) Convert the flowchart for Kruskal's algorithm into code



Ans: Code

```
# include <stdio.h>
int p[10];
int f(int x) { return p[x] == x ? x : p[x] = f(p[x]); }
int main() {
int a[10][10], n, i, j, x, y, m, c=0;
scanf("%d", &n);
for (i=0; i<n; i++) {p[i]=i; for (j=0; j<n; j
++) scanf("%d", &a[i][j]);}
while (c<n-1) { m=999;
 for (i=0; j<n; i++) for (j=i+1; j<n; j++)
if (a[i][j] && a[i][j] <m) m=a[i][j], x=i, y=j;
x = f(x); y = f(y);
if (x!=y) p[x]=y, printf("%d ",m), c++;
 a[x][y] = a[y][x] = 999; }
}
```